Laporan Praktikum Filter Spasial pada Citra Digital (Sesi 5)

SYAHRIZAL HAMDALA

20240040310

1. Pendahuluan

Tujuan. Praktikum ini bertujuan menerapkan beberapa filter spasial pada citra grayscale, yaitu mean, Gaussian, median, dan sharpening (Laplacian). Hasilnya dibandingkan pada dua kondisi: citra asli dan citra yang sengaja diberi noise. Perbandingan dilakukan secara visual dan dengan statistik (nilai minimum, maksimum, rata-rata, standar deviasi, dan kekuatan tepi), untuk menentukan filter yang paling sesuai.

Deskripsi citra. Citra yang dipakai adalah `#Kabinet.png`, berukuran 2025 × 1620 piksel dengan 3 kanal warna (RGB). Citra berupa poster potret seorang pria berjas gelap dengan latar gradasi dari abu-abu terang ke gelap. Citra memuat teks "Koordinator Syahrizal Hamdala", logo di bagian atas, dan deretan ikon media sosial di tepi bawah dengan latar putih. Citra ini cocok untuk percobaan karena memiliki area halus (latar gradasi), tepi tajam (teks, logo, ikon, dan kontur jas), serta detail kecil (teks dan ikon). Dengan begitu efek tiap filter terhadap noise, tepi, dan detail dapat dibandingkan. Citra dikonversi ke grayscale terlebih dahulu agar analisis intensitas lebih sederhana.

2. Metode

2.1 Filter dan ukuran kernel

Filter	Ukuran kernel	Fungsi pustaka	Karakteristik
Mean	3×3 dan 5×5	`scipy.ndimage.convolve`	Semua piksel dalam kernel berbobot sama
Gaussian	3×3 dan 5×5	`scipy.ndimage.convolve`	Bobot terbesar di pusat kernel
Median	3×3 dan 5×5	`scipy.ndimage.median_filter`	Mengambil nilai tengah, non-linear
Sharpening (Laplacian)	3×3	`scipy.ndimage.convolve`	Menambah komponen frekuensi tinggi

Kernel yang digunakan:

$$\text{Mean } 3\times3 = \frac{1}{9} \times \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$$\text{Mean } 5\times5 = \frac{1}{25} \times \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

$$\text{Gaussian } 3\times3 = \frac{1}{16} \times \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 2 & 1 \end{bmatrix}$$

$$\text{Gaussian } 5\times5 = \frac{1}{256} \times \begin{bmatrix} 1 \\ 4 \\ 6 \\ 4 \\ 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 4 & 6 & 4 & 1 \end{bmatrix}$$

$$\text{Sharpening} = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Konvolusi memakai mode tepi `reflect`, dan hasilnya dipotong (*clip*) ke rentang 0–255 lalu dikonversi ke `uint8`.

2.2 Pembuatan noise

Citra bernoise dibuat dari citra grayscale dengan dua jenis noise sekaligus (seed acak = 42 agar hasil dapat diulang):

1. Noise Gaussian dengan rata-rata 0 dan simpangan baku 12.
2. Noise *salt and pepper* sebanyak 2% dari jumlah piksel; separuhnya diset ke 0 (hitam) dan separuhnya ke 255 (putih).

2.3 Metrik evaluasi

- **Min, Max, Mean, Std** dari intensitas piksel. Std dipengaruhi struktur citra, bukan hanya noise.
- **Kekuatan tepi** adalah rata-rata magnitudo gradien Sobel, `mean(√(Gx² + Gy²))`. Nilai besar berarti banyak perubahan intensitas tajam. Pada citra bernoise, nilai ini juga naik karena noise ikut dihitung sebagai "tepi".
- **Patch 5×5** di titik dengan gradien terbesar (baris 1977, kolom 79) untuk melihat perubahan nilai piksel secara langsung.

2.4 Pustaka dan lingkungan

- Bahasa: Python 3
- Pustaka: `imageio` (membaca citra), `NumPy` (operasi array), `SciPy` modul `ndimage` (konvolusi, median filter, Sobel), `Matplotlib` dengan backend `Agg` (menyimpan gambar tanpa jendela tampilan)
- Masukan: `#Kabinet.png`; keluaran: `hasil_filter_spasial.png`, `hasil_filter_spasial_noise.png`, dan `nilai_intensitas_sesi5.txt`

3. Hasil

3.1 Kode utama

Berikut fungsi inti yang menerapkan filter, menghitung statistik, dan menambahkan noise. Kode lengkap yang dapat dijalankan ada di Lampiran.

```
KERNEL_MEAN_3 = np.ones((3, 3), dtype=np.float64) / 9.0
KERNEL_MEAN_5 = np.ones((5, 5), dtype=np.float64) / 25.0
KERNEL_GAUSS_3 = np.outer([1, 2, 1], [1, 2, 1]) / 16.0
KERNEL_GAUSS_5 = np.outer([1, 4, 6, 4, 1], [1, 4, 6, 4, 1]) /
256.0
KERNEL_SHARPEN = np.array([[0, -1, 0], [-1, 5, -1], [0, -1,
0]], dtype=np.float64)


def konvolusi(gray, kernel):
    hasil = ndimage.convolve(gray.astype(np.float64), kernel,
mode="reflect")
    return np.clip(hasil, 0, 255).astype(np.uint8)
```

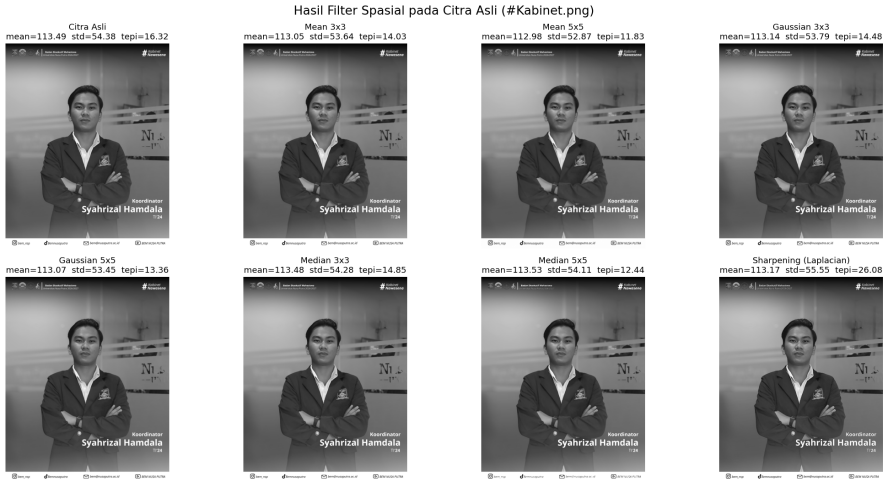
```
def median_filter(gray, ukuran):
    return ndimage.median_filter(gray,
size=ukuran).astype(np.uint8)


def statistik(gray):
    f = gray.astype(np.float64)
    gx = ndimage.sobel(f, axis=1)
    gy = ndimage.sobel(f, axis=0)
    kekuatan_tepi = np.hypot(gx, gy).mean()
    return {"min": int(gray.min()), "max": int(gray.max()),
            "mean": float(f.mean()), "std": float(f.std()),
            "tepi": float(kekuatan_tepi)}


def tambah_noise(gray, seed=42):
    rng = np.random.default_rng(seed)
    noisy = gray.astype(np.float64) + rng.normal(0, 12,
gray.shape)
    jumlah_sp = int(0.02 * gray.size)
    baris = rng.integers(0, gray.shape[0], jumlah_sp)
    kolom = rng.integers(0, gray.shape[1], jumlah_sp)
    noisy[baris[: jumlah_sp // 2], kolom[: jumlah_sp // 2]] =
0
    noisy[baris[jumlah_sp // 2 :], kolom[jumlah_sp // 2 :]] =
255
    return np.clip(noisy, 0, 255).astype(np.uint8)
```

3.2 Perbandingan pada citra asli

Gambar 1. Hasil Filter Spasial pada Citra Asli (`#Kabinet.png`) — `hasil_filter_spasial.png`

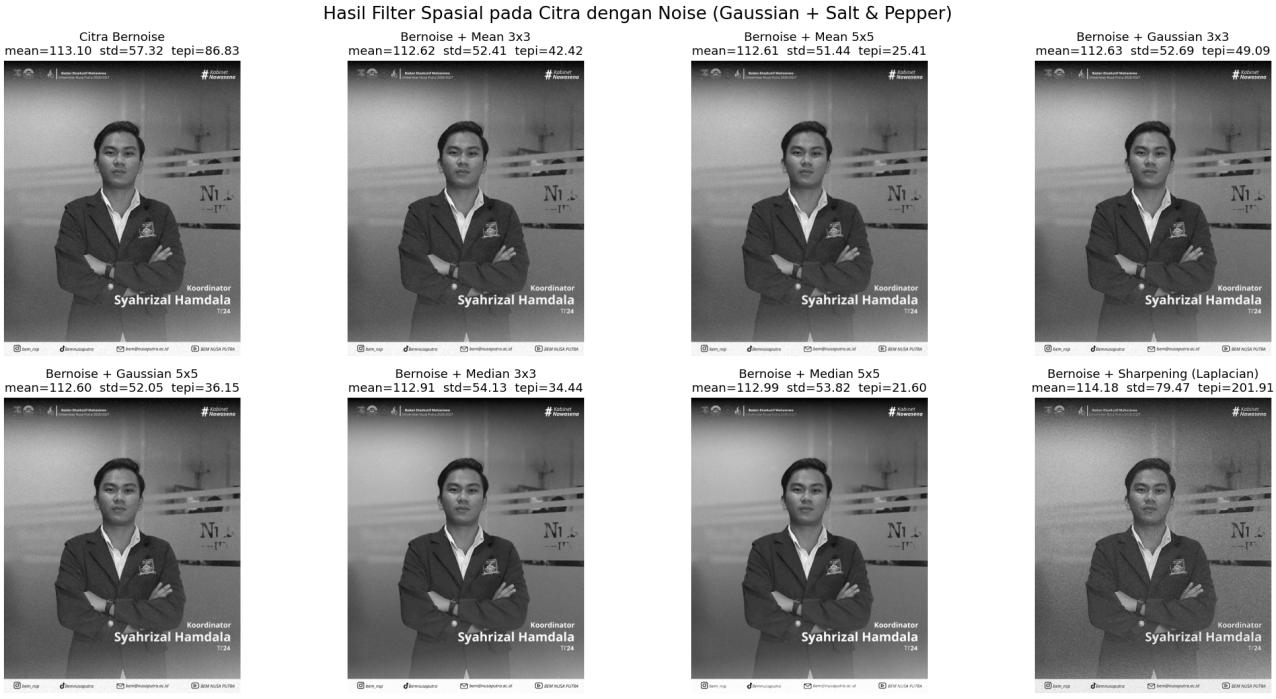


Tabel 1. Statistik citra asli dan hasil filter

Citra	Min	Max	Mean	Std	Kekuatan Tepi
Citra Asli	0	255	113,49	54,38	16,32
Mean 3×3	0	255	113,05	53,64	14,03
Mean 5×5	0	254	112,98	52,87	11,83
Gaussian 3×3	0	255	113,14	53,79	14,48
Gaussian 5×5	0	255	113,07	53,45	13,36
Median 3×3	0	255	113,48	54,28	14,85
Median 5×5	0	255	113,53	54,11	12,44
Sharpening (Laplacian)	0	255	113,17	55,55	26,08

3.3 Perbandingan pada citra bernoise

Gambar 2. Hasil Filter Spasial pada Citra dengan Noise (Gaussian + Salt & Pepper) — `hasil_filter_spasial_noise.png`



Tabel 2. Statistik citra bernoise dan hasil filter

Citra	Min	Max	Mean	Std	Kekuatan Tepi
Citra Bernoise	0	255	113,10	57,32	86,83
Bernoise + Mean 3×3	0	255	112,62	52,41	42,42
Bernoise + Mean 5×5	0	254	112,61	51,44	25,41
Bernoise + Gaussian 3×3	0	255	112,63	52,69	49,09
Bernoise + Gaussian 5×5	0	254	112,60	52,05	36,15
Bernoise + Median 3×3	0	255	112,91	54,13	34,44
Bernoise + Median 5×5	0	255	112,99	53,82	21,60
Bernoise + Sharpening (Laplacian)	0	255	114,18	79,47	201,91

3.4 Contoh nilai intensitas patch 5×5

Patch diambil pada baris 1977, kolom 79, yaitu titik dengan gradien terbesar pada citra asli. Titik ini berada pada perbatasan tajam antara area putih dan area gelap di bagian bawah citra. Baris pertama patch pada tiap filter:

Citra	Baris pertama patch
Citra Asli	255 164 0 0 0
Mean 3×3	224 139 54 0 3
Mean 5×5	189 138 87 42 40
Gaussian 3×3	232 146 41 0 2
Gaussian 5×5	216 142 59 13 14
Median 3×3	255 164 0 0 0
Median 5×5	255 164 0 0 0
Sharpening	255 235 0 0 0

Seluruh patch tersedia pada berkas `nilai_intensitas_sesi5.txt`.

4. Pembahasan

4.1 Pengurangan noise

Pada citra asli, standar deviasi turun dari 54,38 menjadi 53,64 (mean 3×3), 52,87 (mean 5×5), 53,79 (Gaussian 3×3), dan 53,45 (Gaussian 5×5). Penurunan ini kecil karena citra asli memang tidak banyak noise, dan std lebih banyak mencerminkan struktur objek.

Efek yang sebenarnya terlihat pada citra bernoise. Noise menaikkan std dari 54,38 menjadi 57,32 dan melonjakkan kekuatan tepi dari 16,32 menjadi 86,83, kira-kira 5 kali lipat. Pada Gambar 2, noise terlihat sebagai butiran halus di latar dan bintik hitam-putih di area terang. Setelah penyaringan, kekuatan tepi turun menjadi:

Filter	Kekuatan tepi (bernoise)	Std
Median 5×5	21,60	53,82
Mean 5×5	25,41	51,44
Median 3×3	34,44	54,13
Gaussian 5×5	36,15	52,05
Mean 3×3	42,42	52,41
Gaussian 3×3	49,09	52,69

Pada ukuran kernel yang sama, median meredam lebih banyak noise daripada mean dan Gaussian (34,44 berbanding 42,42 dan 49,09 pada 3×3). Penyebabnya, nilai ekstrem 0 atau 255 dari salt and pepper tidak ikut dirata-ratakan oleh median, melainkan digantikan nilai tengah lingkungannya. Pada mean dan Gaussian, nilai ekstrem itu masih ikut menyumbang ke hasil rata-rata. Gaussian meninggalkan lebih banyak sisa noise daripada mean pada ukuran yang sama karena bobot pusatnya lebih besar sehingga penghalusannya lebih ringan.

Std juga memberi petunjuk. Median 3×3 menghasilkan std 54,13, paling dekat dengan citra asli tanpa noise (54,38). Mean dan Gaussian menurunkan std hingga 51–53, lebih rendah dari citra asli, yang menandakan sebagian variasi nyata dari citra ikut hilang akibat blur.

Sharpening bekerja berlawanan: pada citra bernoise std naik menjadi 79,47 dan kekuatan tepi menjadi 201,91 (sekitar 2,3 kali citra bernoise). Pada Gambar 2, panel sharpening tampak paling kasar dan berbintik. Ini membuktikan penajaman memperkuat noise.

4.2 Ketajaman tepi dan detail

Kekuatan tepi citra asli adalah 16,32. Penurunan relatif pada citra asli:

Filter	Kekuatan tepi	Perubahan
Median 3×3	14,85	turun sekitar 9%
Gaussian 3×3	14,48	turun sekitar 11%
Mean 3×3	14,03	turun sekitar 14%
Gaussian 5×5	13,36	turun sekitar 18%
Median 5×5	12,44	turun sekitar 24%
Mean 5×5	11,83	turun sekitar 28%

Sharpening	26,08	naik sekitar 60%

Urutan ini sejalan dengan teori: mean paling mengaburkan tepi karena semua piksel berbobot sama, Gaussian lebih lembut karena bobot pusat besar, dan median 3×3 paling sedikit mengubah tepi. Secara visual pada Gambar 1, teks "Syahrizal Hamdala", ikon di bagian bawah, dan kontur jas paling jelas pada citra asli, median 3×3, dan sharpening, sedangkan pada mean 5×5 dan Gaussian 5×5 tepinya tampak lebih lembut.

Bukti paling langsung ada pada patch 5×5 di tepi tajam. Nilai asli `255 164 0 0 0` dipertahankan persis oleh median 3×3, sedangkan mean 3×3 mengubahnya menjadi `224 139 54 0 3` dan mean 5×5 menjadi `189 138 87 42 40`. Peralihan putih ke hitam yang semula tajam (255 ke 0 dalam dua piksel) melebar menjadi gradasi yang landai. Pada median 5×5, struktur kecil mulai terhapus, misalnya nilai 117 di tengah patch berubah menjadi 164, dan nilai 240 menjadi 242.

4.3 Artefak

- **Blur.** Mean dan Gaussian mengaburkan tepi dan detail halus. Efeknya makin kuat pada kernel 5×5, seperti terlihat pada nilai kekuatan tepi dan perataan transisi pada patch.
- **Hilangnya detail tipis.** Median 5×5 mulai menghapus struktur kecil seperti sudut dan garis tipis (lihat perubahan nilai pada patch di atas).
- **Overshoot dan clipping pada sharpening.** Nilai 164 pada patch menjadi 235, dan nilai 117 menjadi 88, sementara nilai 242 menjadi 255 dan 186 menjadi 0. Perubahan ekstrem ini menandakan penajaman berlebih pada tepi serta pemotongan nilai ke batas 0 dan 255. Pada citra bernoise, rata-rata sharpening naik menjadi 114,18 dari 113,10, kemungkinan karena pemotongan nilai ekstrem noise.
- **Selisih kecil pada rata-rata.** Mean dan Gaussian menghasilkan rata-rata sedikit lebih rendah (sekitar 113,0–113,1 dari 113,49), dan nilai maksimum mean 5×5 menjadi 254. Hal ini kemungkinan terkait konversi `uint8` yang memotong pecahan, bukan membulatkannya.

4.4 Pengaruh parameter

Ukuran kernel adalah parameter yang paling berpengaruh. Kernel yang lebih besar meredam lebih banyak noise, tetapi mengurangi detail. Hal ini terlihat konsisten pada semua filter: kekuatan tepi bernoise turun dari 42,42 ke 25,41 (mean), dari 49,09 ke 36,15 (Gaussian), dan dari 34,44 ke 21,60 (median) saat kernel diperbesar dari 3×3 ke 5×5, sementara kekuatan tepi citra asli ikut turun dari 14,03 ke 11,83, 14,48 ke 13,36, dan 14,85 ke 12,44. Pada Gaussian, distribusi bobot menentukan karakter blur: bobot pusat besar menjaga struktur utama, sedangkan bobot tepi yang lebih besar memperlebar efek penghalusan. Pada sharpening, besarnya koefisien pusat dan nilai negatif di sekelilingnya menentukan kekuatan penajaman; semakin besar, semakin besar pula risiko overshoot dan penguatan noise.

Rata-rata intensitas relatif stabil di semua filter (112,6–114,2). Perubahan utama terjadi pada sebaran intensitas (std) dan hubungan antar-piksel, bukan pada kecerahan keseluruhan.

4.5 Catatan tentang metrik

Pada citra bernoise, kekuatan tepi bercampur antara tepi nyata dan noise, sehingga penurunannya tidak otomatis berarti tepi menjadi lebih kabur. Oleh karena itu, hasil dibaca bersama std,

pengamatan visual, dan patch nilai piksel. Pembandingan dengan citra asli sebagai acuan (misalnya PSNR atau SSIM) tidak dihitung pada praktikum ini.

5. Kesimpulan

1. **Pilihan utama: median 3×3.** Filter ini paling baik menyeimbangkan penghalusan noise dan kelestarian tepi. Pada citra asli, kekuatan tepinya 14,85 (hanya turun sekitar 9% dari 16,32) dengan std 54,28 yang hampir sama dengan aslinya, dan nilai patch di tepi tajam tidak berubah. Pada citra bernoise, median 3×3 menurunkan kekuatan tepi dari 86,83 menjadi 34,44 dan memberi std 54,13, paling dekat dengan citra asli.
2. **Gaussian 3×3** cocok bila noise dominan bersifat Gaussian, karena penghalusannya merata dan tepinya (14,48) masih terjaga cukup baik.
3. **Mean 5×5** tidak disarankan bila ketajaman tepi menjadi prioritas, karena menurunkan kekuatan tepi paling besar (11,83) dan paling mengaburkan transisi tepi.
4. **Median 5×5** menekan noise paling kuat (kekuatan tepi bernoise 21,60), tetapi mulai menghapus detail kecil sehingga hanya layak bila noise sangat berat.
5. **Sharpening** sebaiknya diterapkan pada citra yang sudah bersih (atau setelah proses denoising), bukan pada citra bernoise. Pada citra asli ia menaikkan kekuatan tepi hingga 26,08, tetapi pada citra bernoise ia melipatgandakan noise (kekuatan tepi 201,91) dan menimbulkan clipping serta overshoot di sekitar tepi.

Keterbatasan. Percobaan hanya memakai satu citra dan satu contoh noise (seed 42), sehingga hasil belum tentu berlaku umum. Evaluasi bersandar pada statistik sederhana dan pengamatan visual, tanpa metrik acuan seperti PSNR atau SSIM. Median kurang efektif untuk noise Gaussian murni dan dapat menghilangkan tekstur halus, sedangkan semua filter di sini berukuran tetap dan tidak adaptif terhadap isi citra.

6. Lampiran

Kode lengkap berikut dapat dijalankan langsung. Letakkan `#Kabinet.png` di folder yang sama dengan skrip. Program mencetak statistik ke konsol dan menyimpan tiga berkas keluaran.

```
import imageio.v2 as imageio
import numpy as np
from scipy import ndimage
import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt

NAMA_FILE = "#Kabinet.png"

KERNEL_MEAN_3 = np.ones((3, 3), dtype=np.float64) / 9.0
KERNEL_MEAN_5 = np.ones((5, 5), dtype=np.float64) / 25.0

KERNEL_GAUSS_3 = np.outer([1, 2, 1], [1, 2, 1]) / 16.0
KERNEL_GAUSS_5 = np.outer([1, 4, 6, 4, 1], [1, 4, 6, 4, 1]) /
256.0
```

```
KERNEL_SHARPEN = np.array(
    [
        [0, -1, 0],
        [-1, 5, -1],
        [0, -1, 0],
    ],
    dtype=np.float64,
)


def ke_grayscale(img):
    if img.ndim == 2:
        return img.astype(np.uint8)
    if img.shape[2] == 4:
        img = img[:, :, :3]
    r, g, b = img[:, :, 0], img[:, :, 1], img[:, :, 2]
    return (0.299 * r + 0.587 * g + 0.114 *
b).astype(np.uint8)


def konvolusi(gray, kernel):
    hasil = ndimage.convolve(gray.astype(np.float64), kernel,
mode="reflect")
    return np.clip(hasil, 0, 255).astype(np.uint8)


def mean_filter(gray, ukuran):
    kernel = KERNEL_MEAN_3 if ukuran == 3 else KERNEL_MEAN_5
    return konvolusi(gray, kernel)


def gaussian_filter(gray, ukuran):
    kernel = KERNEL_GAUSS_3 if ukuran == 3 else
KERNEL_GAUSS_5
    return konvolusi(gray, kernel)


def median_filter(gray, ukuran):
    return ndimage.median_filter(gray,
size=ukuran).astype(np.uint8)


def sharpening(gray):
    return konvolusi(gray, KERNEL_SHARPEN)
```

```
def statistik(gray):
    f = gray.astype(np.float64)
    gx = ndimage.sobel(f, axis=1)
    gy = ndimage.sobel(f, axis=0)
    kekuatan_tepi = np.hypot(gx, gy).mean()
    return {
        "min": int(gray.min()),
        "max": int(gray.max()),
        "mean": float(f.mean()),
        "std": float(f.std()),
        "tepi": float(kekuatan_tepi),
    }


def tambah_noise(gray, seed=42):
    rng = np.random.default_rng(seed)
    noisy = gray.astype(np.float64)

    gaussian_noise = rng.normal(0, 12, gray.shape)
    noisy = noisy + gaussian_noise

    jumlah_sp = int(0.02 * gray.size)
    baris = rng.integers(0, gray.shape[0], jumlah_sp)
    kolom = rng.integers(0, gray.shape[1], jumlah_sp)
    noisy[baris[: jumlah_sp // 2], kolom[: jumlah_sp // 2]] =
0
    noisy[baris[jumlah_sp // 2 :], kolom[jumlah_sp // 2 :]] =
255

    return np.clip(noisy, 0, 255).astype(np.uint8)


def terapkan_semua(gray):
    return [
        ("Citra Asli", gray),
        ("Mean 3x3", mean_filter(gray, 3)),
        ("Mean 5x5", mean_filter(gray, 5)),
        ("Gaussian 3x3", gaussian_filter(gray, 3)),
        ("Gaussian 5x5", gaussian_filter(gray, 5)),
        ("Median 3x3", median_filter(gray, 3)),
        ("Median 5x5", median_filter(gray, 5)),
        ("Sharpening (Laplacian)", sharpening(gray)),
    ]
```

```
def tampilkan(hasil, judul_gambar, nama_keluaran):
    fig, axes = plt.subplots(2, 4, figsize=(18, 9))
    for ax, (judul, citra) in zip(axes.ravel(), hasil):
        s = statistik(citra)
        ax.imshow(citra, cmap="gray", vmin=0, vmax=255)
        ax.set_title(
            f"{judul}\nmean={s['mean']:.2f}
std={s['std']:.2f}  "
            f"tepi={s['tepi']:.2f}",
            fontsize=11,
        )
        ax.axis("off")
    fig.suptitle(judul_gambar, fontsize=16)
    plt.tight_layout()
    plt.savefig(nama_keluaran, dpi=120, bbox_inches="tight")
    plt.close(fig)


img = imageio.imread(NAMA_FILE)
gray = ke_grayscale(img)

hasil_asli = terapkan_semua(gray)
tampilkan(
    hasil_asli,
    "Hasil Filter Spasial pada Citra Asli (#Kabinet.png)",
    "hasil_filter_spasial.png",
)

noisy = tambah_noise(gray)
hasil_noise = [("Citra Bernoise", noisy)] + [
    (f"Bernoise + {judul}", citra)
    for judul, citra in terapkan_semua(noisy)[1:]
]
tampilkan(
    hasil_noise,
    "Hasil Filter Spasial pada Citra dengan Noise (Gaussian +
Salt & Pepper)",
    "hasil_filter_spasial_noise.png",
)

f = gray.astype(np.float64)
gx = ndimage.sobel(f, axis=1)
gy = ndimage.sobel(f, axis=0)
gradien = np.hypot(gx, gy)
```

```
baris, kolom = np.unravel_index(np.argmax(gradien),
gradien.shape)
baris = int(np.clip(baris, 2, gray.shape[0] - 3))
kolom = int(np.clip(kolom, 2, gray.shape[1] - 3))
patch_asli = gray[baris - 2 : baris + 3, kolom - 2 : kolom +
3]

lines = []
lines.append(f"Citra sumber: {NAMA_FILE}")
lines.append(f"Ukuran citra: {img.shape}")
lines.append("")
lines.append("=" * 88)
lines.append("STATISTIK CITRA ASLI DAN HASIL FILTER")
lines.append("=" * 88)
lines.append(
    f"{'Citra':<26}{'Min':>6}{'Max':>6}{'Mean':>10}
{'Std':>10}{'Kekuatan Tepi':>16}"
)
lines.append("-" * 88)
for judul, citra in hasil_asli:
    s = statistik(citra)
    lines.append(
        f"{judul:<26}{s['min']:>6}{s['max']:>6}
{s['mean']:>10.2f}"
        f"{s['std']:>10.2f}{s['tepi']:>16.2f}"
    )

lines.append("")
lines.append("=" * 88)
lines.append("STATISTIK CITRA BERNOISE DAN HASIL FILTER")
lines.append("=" * 88)
lines.append(
    f"{'Citra':<26}{'Min':>6}{'Max':>6}{'Mean':>10}
{'Std':>10}{'Kekuatan Tepi':>16}"
)
lines.append("-" * 88)
for judul, citra in hasil_noise:
    s = statistik(citra)
    lines.append(
        f"{judul:<26}{s['min']:>6}{s['max']:>6}
{s['mean']:>10.2f}"
        f"{s['std']:>10.2f}{s['tepi']:>16.2f}"
    )

lines.append("")
```

```
lines.append("=" * 88)
lines.append(f"CONTOH NILAI INTENSITAS PATCH 5x5 (baris
{baris}, kolom {kolom})")
lines.append("=" * 88)
for judul, citra in hasil_asli:
    patch = citra[baris - 2 : baris + 3, kolom - 2 : kolom +
3]
    lines.append(f"{judul}:")
    lines.append(np.array2string(patch, max_line_width=200))
    lines.append("")

teks = "\n".join(lines)
print(teks)
with open("nilai_intensitas_sesi5.txt", "w",
encoding="utf-8") as file:
    file.write(teks)
```